Übung 1

Algorithmische Grundbausteine

Programmierung 1 · WS 2026/27
Bachelor Cyber Security & Privacy · André Kless, M.Sc.

Ziel: Einfache algorithmische Probleme selbstständig lösen: Eingabe, Verarbeitung und Ausgabe bestimmen, Anweisungen kombinieren und Lösungen nachvollziehen.

Arbeitsweise: Bearbeiten Sie die Aufgaben auf Papier oder digital. Java ist nicht erforderlich. Deklarieren Sie Variablen ausdrücklich, ohne Typangaben. Nutzen Sie die Pseudocode-Schreibweise der Vorlesung, z. B. `DEKLARIERE`, `SETZE ... AUF ...`, `LIES ... EIN` und `GIB ... AUS`.

Reihenfolge: Basic (1-4) festigt einzelne Bausteine. Combined (5-8) kombiniert sie. Challenge (9-10) vertieft das selbstständige Problemlösen und kann zu Hause fortgesetzt werden.

01 Kosten für die Lerngruppe | Basic

Eine Lerngruppe bestellt Hefte. Ein Heft kostet 3 Euro, die Lieferung einmalig 4 Euro. Eingabe ist die Anzahl bestellter Hefte, eine positive ganze Zahl. Ausgabe ist der Gesamtpreis in Euro.

a) Benennen Sie Eingabe, Verarbeitung und Ausgabe. Welche Variablen benötigen Sie?

b) Entwerfen Sie einen Pseudocode mit Deklaration, Eingabe, Zuweisung und Ausgabe. Stellen Sie dieselbe Lösung als Struktogramm dar.

c) Prüfen Sie Ihre Lösung für 1 und für 5 Hefte. Notieren Sie jeweils den erwarteten Gesamtpreis vor dem Durchspielen.

02 Zwei Werte vertauschen | Basic

Nach der Eingabe enthält `a` den Wert 4 und `b` den Wert 9. Jemand versucht, die Werte so zu vertauschen:

```
SETZE a AUF b
SETZE b AUF a
```

a) Verfolgen Sie beide Zuweisungen in einer Trace-Tabelle mit den Spalten Anweisung, `a` und `b`. Tragen Sie auch den Ausgangszustand ein. Warum funktioniert der Versuch nicht?

b) Entwickeln Sie eine korrekte Lösung mit einer zusätzlichen Variable. Schreiben Sie den vollständigen Pseudocode einschließlich Deklarationen, Eingaben und Ausgabe beider Werte.

c) Prüfen Sie die Lösung auch für zwei gleiche Eingabewerte.

Basic: auswählen und Fälle unterscheiden

03 Die größere Zahl | Basic

Eingaben sind zwei ganze Zahlen `a` und `b`. Der Algorithmus soll die größere Zahl ausgeben. Sind beide gleich, soll er diesen gemeinsamen Wert genau einmal ausgeben.

a) Beschreiben Sie die Ausgabe für jeden möglichen Vergleich der beiden Zahlen.

b) Entwerfen Sie Pseudocode und ein Struktogramm mit einer Selektion. Nutzen Sie keine fertige Maximum-Funktion.

c) Prüfen Sie die Eingaben (7, 3), (3, 7), (5, 5) und (-2, -8). Begründen Sie, warum diese Fälle sinnvoll sind.

04 Ein Menü für den Lernraum | Basic

Ein Terminal liest eine ganze Zahl als Auswahl ein. Es soll genau eine passende Meldung ausgeben:

Auswahl	Ausgabe
1	„Lernraum buchen“
2	„Buchung anzeigen“
3	„Buchung stornieren“
Jede andere ganze Zahl	„Ungültige Auswahl“

a) Lösen Sie die Aufgabe mit einer Mehrfachselektion in Pseudocode. Es geht nur um die Meldung, nicht um die tatsächliche Buchung.

b) Übertragen Sie den Algorithmus in ein Struktogramm. Kennzeichnen Sie den Fall für alle übrigen Eingaben.

c) Wählen Sie Testeingaben, mit denen Sie jeden Zweig mindestens einmal prüfen. Notieren Sie die erwarteten Ausgaben.

Prüfen Sie Ihre Lösungen

Lässt sich jeder Schritt eindeutig ausführen? Wird in jedem Fall genau die verlangte Ausgabe erzeugt? Kann jemand anderes Ihrer Lösung ohne zusätzliche mündliche Erklärung folgen?

Combined: Wiederholungen gezielt einsetzen

05 Zahlen von 1 bis n addieren | Combined

Eingabe ist eine ganze Zahl `n` mit `n >= 0`. Gesucht ist die Summe der Zahlen von 1 bis n. Für `n = 0` soll die Ausgabe 0 sein. Nutzen Sie wiederholte Addition, keine Summenformel.

a) Bestimmen Sie die benötigten Variablen und ihre Anfangswerte. Entwickeln Sie eine Lösung mit `SOLANGE` in Pseudocode und als Struktogramm.

b) Erstellen Sie für `n = 4` eine Trace-Tabelle. Tragen Sie vor jeder Bedingungsprüfung den Zähler, die bisherige Summe und das Ergebnis der Bedingung ein, auch bei der letzten Prüfung.

c) Prüfen Sie `n = 0` und `n = 1`. Erklären Sie, weshalb die Schleife bei `n = 0` keinen Durchlauf benötigt.

d) Formulieren Sie eine zweite Lösung mit `WIEDERHOLE n-MAL`. Was muss dabei innerhalb des Schleifenrumpfs geschehen?

06 Eine gültige Gruppengröße einlesen | Combined

Für eine Lerngruppe sind 2 bis 6 Personen zugelassen, einschließlich beider Grenzen. Es werden ganze Zahlen eingegeben. Der Algorithmus soll so lange erneut einlesen, bis eine gültige Gruppengröße vorliegt. Anschließend gibt er die akzeptierte Größe aus.

a) Formulieren Sie je eine Bedingung dafür, dass die Eingabe gültig ist und dass erneut eingelesen werden muss. Sie dürfen Bedingungen mit `UND` bzw. `ODER` verknüpfen.

b) Entwerfen Sie Pseudocode und ein Struktogramm mit `WIEDERHOLE ... SOLANGE`. Verwenden Sie keine künstliche erste Eingabe und keinen zusätzlichen Abbruchbefehl.

c) Verfolgen Sie die Eingabefolge 0, 7, 4. Welche Bedingung wird wann geprüft und welche Zahl wird am Ende ausgegeben?

d) Prüfen Sie die einzelnen Eingaben 2 und 6. Begründen Sie, warum der Schleifenrumpf auch bei einer sofort gültigen Eingabe einmal ausgeführt wird.

Auf die Bedingung kommt es an

Bei beiden Schleifenformen ist eine wahre Bedingung der Grund zum Weiterlaufen. `SOLANGE` prüft vor dem Rumpf, `WIEDERHOLE ... SOLANGE` danach. Formulieren Sie die Fortsetzungsbedingung bewusst.

Combined: Bausteine verschachteln und Fehler finden

07 Bestellungen aufsummieren | Combined

Zuerst wird eine nichtnegative ganze Zahl `n` eingelesen. Danach folgen n Bestellungen, jeweils als positive ganze Anzahl Hefte. Ab 10 Heften kostet ein Heft 2 Euro, sonst 3 Euro. Der Preis gilt jeweils für alle Hefte dieser Bestellung. Versandkosten gibt es hier nicht.

a) Entwerfen Sie Pseudocode: Lesen Sie die Bestellungen einzeln ein, berechnen Sie deren Preise und geben Sie am Ende die Gesamtkosten aller Bestellungen aus. Für `n = 0` sollen die Gesamtkosten 0 sein.

b) Zeichnen Sie das Struktogramm. Markieren Sie die Sequenz, die Iteration und die darin verschachtelte Selektion.

c) Prüfen Sie `n = 3` mit den Bestellmengen 2, 10 und 4. Notieren Sie je Bestellung den Stückpreis, den Bestellpreis und die bisherige Gesamtsumme. Prüfen Sie zusätzlich eine Bestellung mit 9 Heften.

08 Eine Schleife prüfen und korrigieren | Combined

Dieser Algorithmus soll wie in Aufgabe 5 die Summe von 1 bis n ausgeben. n ist eine nichtnegative ganze Zahl. Der Algorithmus enthält einen Fehler.

```
DEKLARIERE n, zaehler, summe
LIES n EIN
SETZE zaehler AUF 1
SETZE summe AUF 0
SOLANGE zaehler < n:
  SETZE summe AUF summe + zaehler
  SETZE zaehler AUF zaehler + 1
ENDE SOLANGE
GIB summe AUS
```

a) Erstellen Sie für `n = 4` eine Trace-Tabelle. Nutzen Sie die folgende Struktur und ergänzen Sie alle Prüfungen einschließlich der letzten.

b) Vergleichen Sie tatsächliche und verlangte Ausgabe. Benennen und korrigieren Sie die fehlerhafte Anweisung.

c) Prüfen Sie die korrigierte Fassung für `n = 0`, `n = 1` und `n = 4`.

Prüfung Nr.	zaehler	summe	zaehler < n
1			
…			

09 Division durch wiederholtes Subtrahieren | Challenge

Eingaben sind eine nichtnegative ganze Zahl `a` und eine positive ganze Zahl `b`. Gesucht sind der ganzzahlige Quotient `q` und der Rest `r`: Es gilt `a = q * b + r` und `0 <= r < b`.

Verwenden Sie für Berechnungen nur Addition und Subtraktion. Kombinieren Sie diese mit Variablen, Vergleichen, Ein- und Ausgabe sowie den Grundbausteinen. Division, Restoperator und fertige Funktionen sind nicht erlaubt.

a) Entwickeln Sie einen Algorithmus in Pseudocode und als Struktogramm. Die Eingabewerte gelten als gültig.

b) Verfolgen Sie Ihre Lösung für `a = 17` und `b = 5` in einer Trace-Tabelle. Prüfen Sie außerdem (3, 5), (15, 5) und (0, 5).

c) Begründen Sie, weshalb die Wiederholung endet und weshalb der Rest am Ende kleiner als b ist.

10 Welches Team gewinnt? | Challenge

Ein Wettbewerb hat n Teams mit `n >= 1`. Zuerst wird n eingelesen. Danach werden für Team 1 bis Team n jeweils die Punkte aus genau drei Runden eingelesen. Jede Punktzahl ist eine ganze Zahl von 0 bis 10; alle Eingaben sind gültig.

Es gewinnt das Team mit der höchsten Summe seiner drei Runden. Bei Gleichstand gewinnt das Team mit der kleineren Nummer. Geben Sie die Nummer des Siegerteams und seine Gesamtpunktzahl aus. Die Rundenergebnisse sollen nicht in einem Feld gespeichert werden.

a) Entwerfen Sie Pseudocode mit zwei verschachtelten Iterationen und einer passenden Selektion. Entscheiden Sie, welche Werte Sie während des Einlesens speichern müssen.

b) Zeichnen Sie das Struktogramm. Zeigen Sie, wo die Teilsumme für jedes neue Team zurückgesetzt wird.

c) Prüfen Sie `n = 3` mit den Punkten (4, 3, 2), (2, 5, 2) und (1, 1, 1). Prüfen Sie zusätzlich `n = 1` mit (0, 0, 0) und einen Fall, in dem erst das letzte Team gewinnt.

d) Erläutern Sie, wie Ihre Lösung den Gleichstand behandelt und weshalb sie auch bei ausschließlich null Punkten einen Sieger ausgibt.